

Informe Técnico: Base de Datos NoSQL con Índice AVL

Arquitectura, Justificación y Operaciones

Resumen

Sistema NoSQL en Java que combina árbol AVL (índice en memoria) con persistencia JSON. Índice AVL garantiza operaciones $O(\log n)$ para búsqueda por ID, con auto-balanceo. Arquitectura en 3 capas: modelo, índice, persistencia.

1 Arquitectura del Sistema

1.1 Componentes Principales

1. Modelo (JsonDocument):

```
public class JsonDocument {  
    private Integer id;           // Clave primaria  
    private JsonNode data;       // JSON flexible  
}
```

2. Índice AVL (AVLTree):

- Nodos con clave, valor, altura y factor de balance
- Auto-balanceo mediante rotaciones simples/dobles
- Altura máxima: $h \leq 1,44\log_2(n + 2)$

3. Controlador (DatabaseController):

```
public class DatabaseController {  
    private AVLTree<Integer, JsonDocument> index;  
    private File jsonFile;  
    // CRUD + sincronización automática  
}
```

4. Flujo de Datos:

1. Operación CRUD en controlador
2. Actualización del árbol AVL en memoria
3. Persistencia inmediata a archivo JSON

1.2 Ventajas Arquitectónicas

- Separación clara: Índice en memoria vs. almacenamiento en disco
- Sincronización automática: Coherencia garantizada
- Flexibilidad JSON: Esquema dinámico para datos
- Ordenamiento natural: Claves ordenadas en árbol

2 Justificación del Árbol AVL

2.1 Comparativa de Estructuras

Estructura	Ventajas	Desventajas	Aptitud
AVL	Balance estricto, búsquedas $O(\log n)$ garantizadas, óptimo memoria	Más rotaciones en escrituras	Alta - Lecturas frecuentes
Rojo-Negro	Menos rotaciones	Búsquedas más lentas, complejo	Media
B/B+ Tree	Óptimo para disco	Sobrecarga memoria	Baja
Hash Table	$O(1)$ promedio	Sin orden, colisiones	Media

Cuadro 1: Comparativa de estructuras de índice

2.2 Justificación Técnica

1. Para el Dominio Específico:

- **Clave primaria ordenada:** Necesaria para recorridos secuenciales
- **Ratio 70/30:** 70 % lecturas (findById) vs. 30 % escrituras
- **Memoria suficiente:** Índice completo en RAM
- **Tamaño moderado:** Hasta $\sim 10^5$ documentos

2. Análisis Matemático:

- Para 10,000 documentos:

$$h_{AVL} \leq 1,44 \log_2(10002) \approx 19 \text{ niveles}$$

$$h_{RojNegro} \leq 2 \log_2(10001) \approx 27 \text{ niveles}$$

- AVL $\sim 30\%$ más bajo = menos comparaciones

3. Complejidad Garantizada:

Operación	AVL	Lista Ordenada
findById	$O(\log n)$	$O(\log n)$
insert/update	$O(\log n)$	$O(n)$
delete	$O(\log n)$	$O(n)$
getAll	$O(n)$	$O(1)$

4. Balance Estricto:

- Factor balance $\in \{-1, 0, 1\}$
- Garantiza rendimiento consistente
- Rotaciones automáticas en inserciones/eliminaciones

2.3 Ventajas sobre Alternativas

vs. Árbol Rojo-Negro:

- AVL tiene altura menor $\Rightarrow$ búsquedas más rápidas
- Implementación más sencilla
- Ideal para más lecturas que escrituras

vs. Tabla Hash:

- AVL mantiene orden $\Rightarrow$ `getAllDocuments()` ordenado
- Sin problemas de colisiones o redimensionamiento

vs. Árbol B+:

- AVL optimizado para memoria principal
- B+ diseñado para E/S disco (no necesario aquí)

3 Ejemplos de Operaciones

3.1 1. Inserción con Balanceo Automático

Caso: Inserción 1, 2, 3

```
db.save(new JsonDocument(1, data1)); // Raíz : 1
db.save(new JsonDocument(2, data2)); // 1 derecha : 2
db.save(new JsonDocument(3, data3)); // Desbalance ! Rotación
```

Proceso:

1. Insertar 1: Raíz = 1, altura = 1
2. Insertar 2: 1→derecha=2, balance(1) = -1
3. Insertar 3: 1→2→derecha=3, balance(1) = -2
4. **Rotación izquierda** en nodo 1:
 - Nueva raíz: 2
 - Hijos: 1 (izq), 3 (der)

Resultado:

```

      2
     /\
    1  3

```

3.2 2. Búsqueda por ID ($O(\log n)$)

```

public Optional<JsonDocument> findById(Integer id) {
    return index.search(id); // Búsqueda binaria en árbol
}

// Uso:
Optional<JsonDocument> doc = db.findById(42);
if (doc.isPresent()) {
    // Encontrado en ~log2(n) pasos
}

```

Eficiencia: Para 10,000 documentos:

- Máximo: 19 comparaciones (altura del árbol)
- Búsqueda lineal: hasta 10,000 comparaciones
- Mejora: $\sim 500\times$ más rápido

3.3 3. Búsqueda por Campo JSON ($O(n)$)

```

public List<JsonDocument> findByField(String field, String value) {
    return findByPredicate(doc -> {
        JsonNode data = doc.getData();
        return data != null &&
            data.get(field) != null &&
            data.get(field).asText().contains(value);
    });
}

// Buscar usuarios Gmail:
List<JsonDocument> gmailUsers =
    db.findByField("email", "gmail.com");

```

Limitación: Escaneo completo $O(n)$ al no tener índice secundario.

3.4 4. Eliminación con Re-balanceo

Casos:

- **Nodo hoja:** Eliminar y balancear hacia arriba
- **Nodo con un hijo:** Reemplazar por hijo
- **Nodo con dos hijos:** Buscar sucesor inorden

```

public boolean deleteById(Integer id) {
    if (!index.contains(id)) return false;
    index.delete(id); // Elimina y re-balancea
    saveToFile();     // Persiste cambios
    return true;
}

```

3.5 5. Ejemplo Integrado

```

// 1. Inicializar
DatabaseController db = new DatabaseController("data.json");

// 2. Insertar documentos
db.save(new JsonDocument(1,
    "{\"nombre\":\"Ana\",\"edad\":30}"));
db.save(new JsonDocument(2,
    "{\"nombre\":\"Juan\",\"edad\":25}"));

// 3. Buscar por ID (r pido)
Optional<JsonDocument> ana = db.findById(1);

```

```
// 4. Buscar por campo (lento)
List<JsonDocument> mayores = db.findByPredicate(
    doc -> doc.getData().get("edad").asInt() > 26
);

// 5. Eliminar
db.deleteById(2);

// 6. Obtener todos ordenados
List<JsonDocument> todos = db.getAllDocuments();
```

4 Limitaciones y Mejoras

4.1 Limitaciones Actuales

- **Persistencia $O(n)$** : Guarda archivo completo en cada operación
- **Índice único**: Solo por ID, no por campos JSON
- **Memoria**: Todo el índice en RAM limita escalabilidad
- **Concurrencia**: Sin soporte para acceso simultáneo

4.2 Mejoras Propuestas

1. Corto Plazo:

- **Persistencia por lotes**: Acumular cambios y guardar periódicamente
- **Índices secundarios**: AVL adicionales para campos frecuentes
- **Cache LRU**: Para documentos más accedidos

2. Largo Plazo:

- **Migración a B+Tree**: Para millones de documentos
- **Sharding**: Distribuir datos por rangos de ID
- **Replicación**: Para alta disponibilidad

3. Optimización Persistencia:

```
// Propuesta: Guardar solo cambios
private List<JsonDocument> pendingChanges;

public void save(JsonDocument doc) {
    index.insert(doc.getId(), doc);
    pendingChanges.add(doc);
    if (pendingChanges.size() >= BATCH_SIZE) {
        saveBatchToFile(); // Append en lugar de overwrite
    }
}
```

Conclusión

El árbol AVL es elección óptima para este sistema NoSQL por: (1) garantía $O(\log n)$ en búsquedas por ID, (2) ordenamiento natural para recorridos, (3) auto-balanceo que mantiene eficiencia, y (4) simplicidad de implementación vs. alternativas. Limitado por persistencia $O(n)$ y falta de índices secundarios, pero adecuado para cargas moderadas (hasta $\sim 10^5$ documentos) con predominio de lecturas.